ECE 315

Steven Knudsen, PhD, PEng

Lecture 28 :

- Interrupts and Events

Last Lecture

- Using DMA to capture real world data

This Lecture

- Interrupts and Events

Learning Objectives

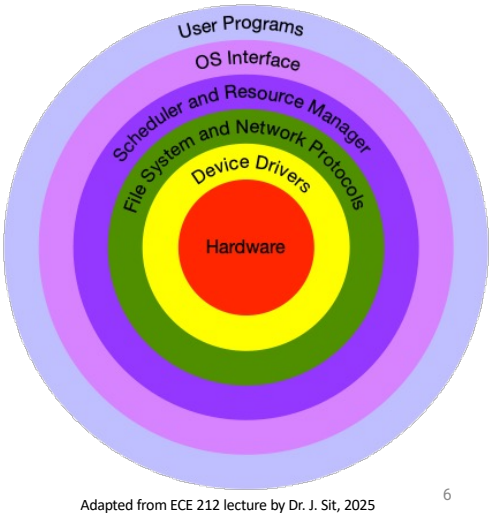
- Explain the role of interrupts in real-time embedded systems
- Differentiate between FreeRTOS tasks, events, and interrupts,
- Describe the structure and purpose of Interrupt Service Routines (ISRs)
  - How interrupts suspend normal task execution and transfer control to an ISR
- Identify common hardware sources of interrupts
- Explain why ISRs must be kept short and non-blocking
- Determine when interrupts should and should not be used
- Define FreeRTOS events and event groups

RTOS and Events

- We've seen that an RTOS is a specialized kind of operating system, meant for embedded applications where what matters is
  - Exact timing
  - Latency, aka responsiveness
  - Performance
  - Dealing with constrained resources
  - Cost
- Of course, performance always matters
- From lecture 4 we have a layered model for an RTOS (aka "onion")

Layer Model of an OS (Lecture 4)

- Scheduler coordinates CPU access by higher layers to OS resources
- It decides which program (aka ***Task***) gets CPU time and when
  - Ideally ensures that all tasks get some time
  - Very complex problem on personal or server computer running many tasks concurrently
  - Somewhat simpler for embedded systems, but complicated by hard-real time requirements

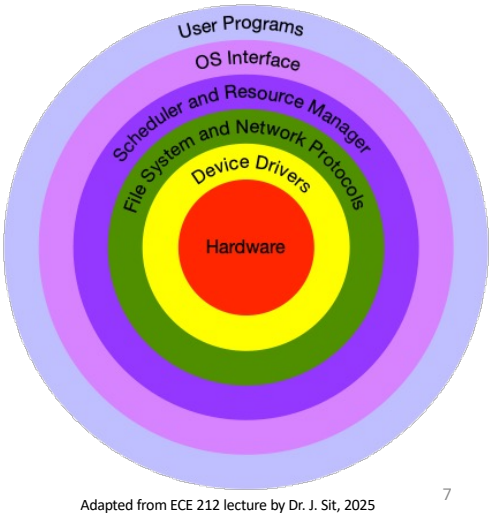


Adapted from ECE 212 lecture by Dr. J. Sit, 2025

Here we are most interested in the idea of a task

Layer Model of an OS (Lecture 4)

- Resource manager monitors and manages the system's resources
  - May include the scheduler functions…
  - Decides which tasks get which resources, like memory, file space, and access to input/output devices.
- FreeRTOS gives us some tools, but we have to implement our own monitoring and management



UNIVERSITY OF ALBERTA

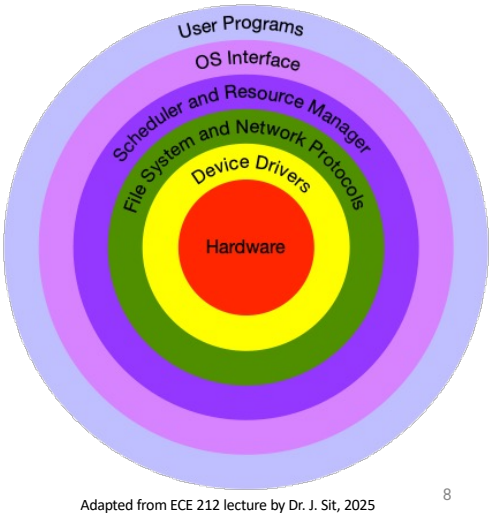
Adapted from ECE 212 lecture by Dr. J. Sit, 2025

7

With FreeRTOS we are responsible for resource monitoring

Layer Model of an OS (Lecture 4)

- The OS Interface provides User Programs access to common OS functions
  - Task definition and management
  - Resource management
  - Scheduler definition and management



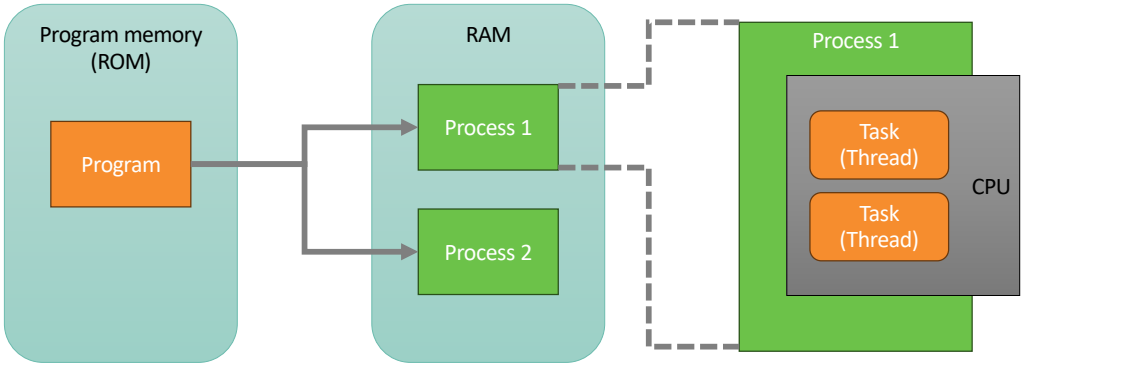
UNIVERSITY OF ALBERTA

Adapted from ECE 212 lecture by Dr. J. Sit, 2025

8

We can reuse the diagram from Lecture 3 and focus on the OS

Processes and Threads (Lecture 4)




```
Process == instance of Program
Process == organization of
           Program in RAM
```

```
Task/Thread == single path of
               execution
Task/Thread is a scheduling entity
```

UNIVERSITY OF ALBERTA

https://www.rtos.be/2013/08/programs-processes-and-threads-part-2/ 9

For context, remember that a process is an instance of a program and that a process will have one or more tasks or threads.

A task is a unit of code that does something, often self-contained, but sometimes tasks cooperate to accomplish something

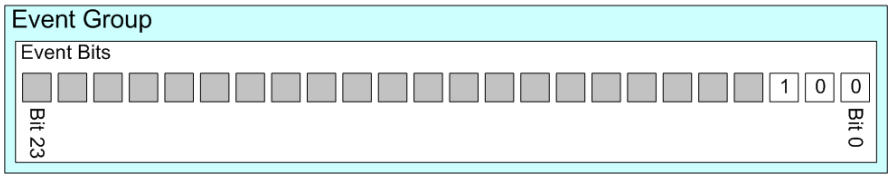
Event Definition

- In FreeRTOS, an event is a kernel-managed notification mechanism used to signal that one or more conditions have occurred, allowing tasks to synchronize or react without busy-waiting
- ***Event Bits*** are used to indicate if an event has occurred or not, e.g.
  - Define a bit (or flag) that means "A message has been received" when set to `1`, and "there are no messages waiting to be processed" when set to `0`
- An ***Event Group*** is a set of event bits
  - Individual event bits within an event group are referenced by bit number, e.g.,
  - The event bit that means "A message has been received" might be bit `0` within an event group

From https://freertos.org/Documentation/02-Kernel/02-Kernel-features/06-Event-groups

Event Definition cont'd

- Event groups referenced by variables of type `EventGroupHandle_t`
  - There are `8` bits in a group if `configUSE_16_BIT_TICKS` is set to `1`, or
  - There are `24` bits if `configUSE_16_BIT_TICKS` is set to `0`
- A single variable of type `EventBits_t` stores the bits
- Example is 24-bit event group with 3 bits used, bit 2 is set to 1.



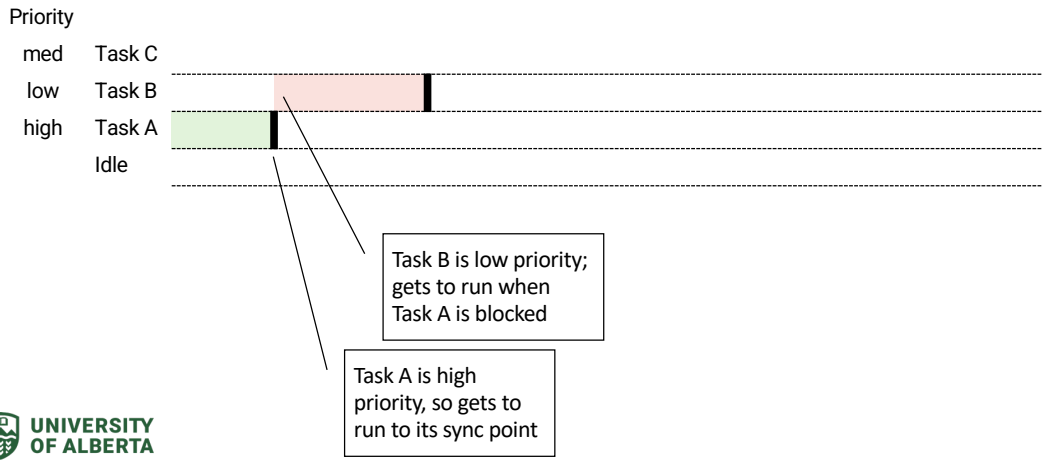
UNIVERSITY OF ALBERTA

From https://freertos.org/Documentation/02-Kernel/02-Kernel-features/06-Event-groups 11

How to use Events

- Event group API functions lets tasks,
  - set one or more event bits within an event group,
  - clear one or more event bits within an event group, and
  - ***pend*** (enter the Blocked state so the task does not consume any processing time) to wait for a set of one or more event bits to become set
- Event groups can also be used to synchronize tasks, creating what is often referred to as a task 'rendezvous'.
  - A task synchronization/rendezvous point is code line where a task waits in the Blocked state (not consuming any CPU time) until all the other tasks taking part in the synchronization also reached their synchronization point

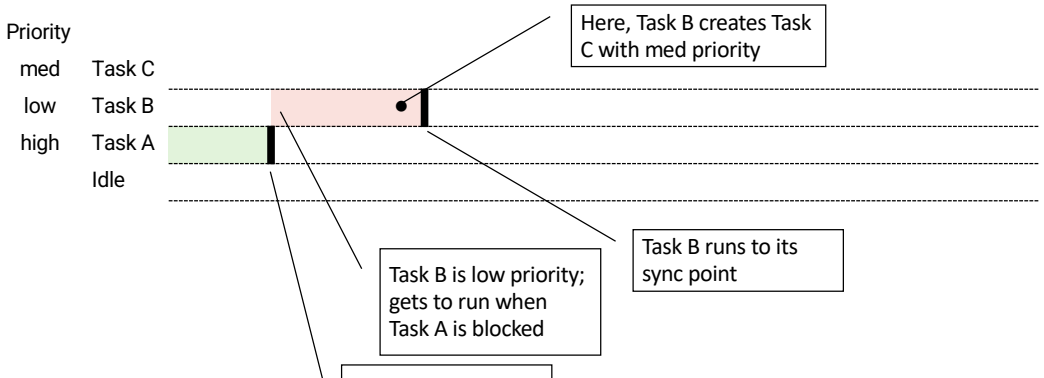
Synchronization/Rendezvous Timing



UNIVERSITY OF ALBERTA

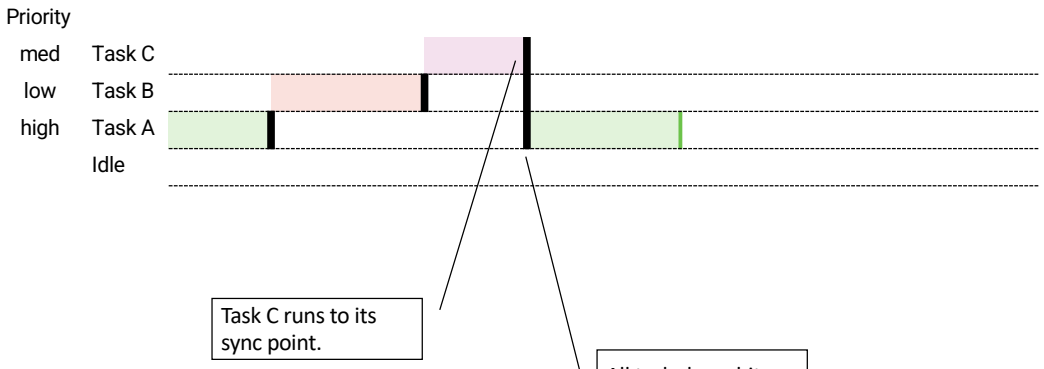
13

Synchronization/Rendezvous Timing



UNIVERSITY OF ALBERTA

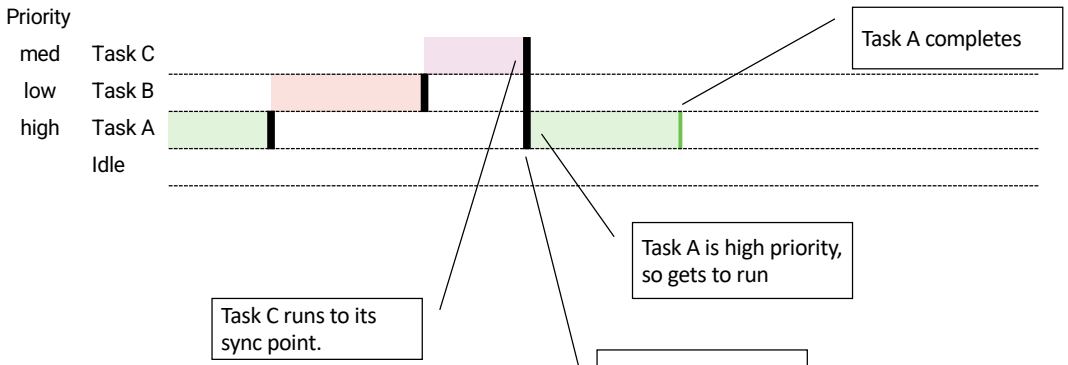
Synchronization/Rendezvous Timing



UNIVERSITY OF ALBERTA

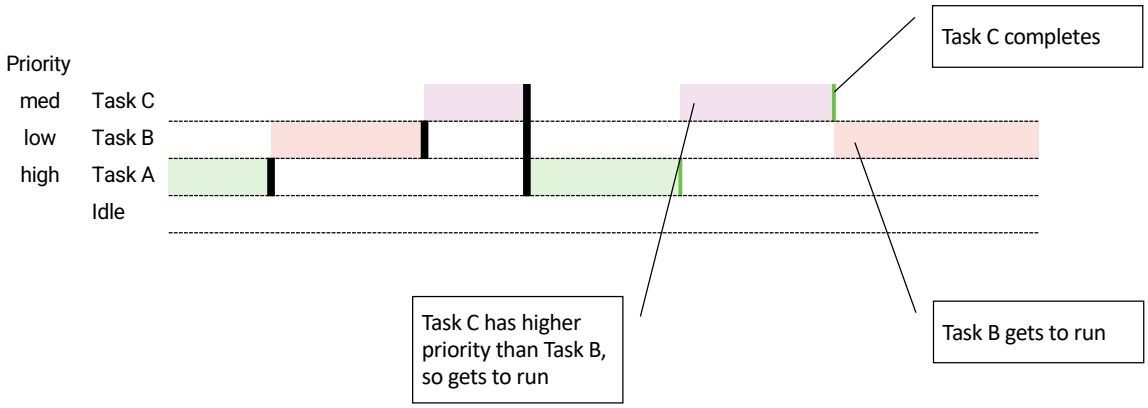
15

Synchronization/Rendezvous Timing



UNIVERSITY OF ALBERTA

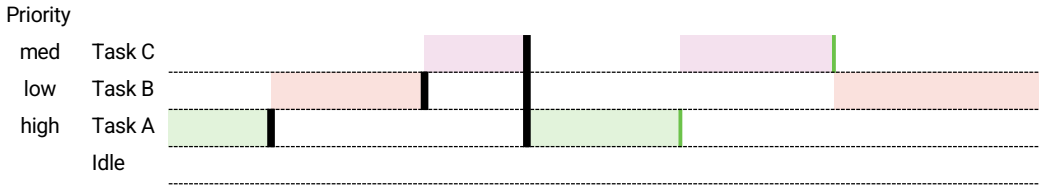
Synchronization/Rendezvous Timing



UNIVERSITY OF ALBERTA

17

Synchronization/Rendezvous Timing



- In this example, it may be that Tasks A and B need what Task C produces, but C can wait until A and B get some work done
  - E.g. Task A might query a peripheral known to take N milliseconds to respond. In the meantime, A and B can do useful work

Synchronization/Rendezvous API

```
1  EventBits_t xEventGroupWaitBits(
2                        const EventGroupHandle_t xEventGroup,
3                        const EventBits_t uxBitsToWaitFor,
4                        const BaseType_t xClearOnExit,
5                        const BaseType_t xWaitForAllBits,
6                        TickType_t xTicksToWait );
```

`xEventGroupWaitBits()`

- Read bits within an event group, optionally entering the Blocked state (with a timeout) to wait for a bit or group of bits to become set
- Must set one or more bits in `uxBitsToWaitFor`
- If `xWaitForAllBits` is true, then this will unblock only when all bits defined in `uxBitsToWaitFor` are set or the call times out

Synchronization/ Rendezvous API

- `xEventGroupWaitBits` returns if either bit is set or `xTicksToWait` expires
- Can choose to do different things depending on what happened
- Other tasks have access to the event group (`xEventGroup`)

UNIVERSITY OF ALBERTA

```c
#define BIT_0      ( 1 << 0 )
#define BIT_4      ( 1 << 4 )

void aFunction( EventGroupHandle_t xEventGroup )
{
    EventBits_t uxBits;
    const TickType_t xTicksToWait = 100 / portTICK_PERIOD_MS;

    /* Wait a maximum of 100ms for either bit 0 or bit 4 to be set within
       the event group. Clear the bits before exiting. */
    uxBits = xEventGroupWaitBits(
                xEventGroup,   /* The event group being tested. */
                BIT_0 | BIT_4, /* The bits within the event group to wait for. */
                pdTRUE,        /* BIT_0 & BIT_4 should be cleared before returning. */
                pdFALSE,       /* Don't wait for both bits, either bit will do. */
                xTicksToWait );/* Wait a maximum of 100ms for either bit to be set. */

    if( ( uxBits & ( BIT_0 | BIT_4 ) ) == ( BIT_0 | BIT_4 ) )
    {
        /* xEventGroupWaitBits() returned because both bits were set. */
    }
    else if( ( uxBits & BIT_0 ) != 0 )
    {
        /* xEventGroupWaitBits() returned because just BIT\_0 was set. */
    }
    else if( ( uxBits & BIT_4 ) != 0 )
    {
        /* xEventGroupWaitBits() returned because just BIT\_4 was set. */
    }
    else
    {
        /* xEventGroupWaitBits() returned because xTicksToWait ticks passed
           without either BIT\_0 or BIT\_4 becoming set. */
    }
}
```

From https://www.freertos.org/Documentation/02-Kernel/04-API-references/12-Event-groups-or-flags/05-xEventGroupSetBits

Interrupts

- Asynchronous signal from hardware
  - E.g., a GPIO set as an input changes state
- Temporarily suspends normal code execution
- Transfers control to an Interrupt Service Routine (ISR)
- Used for time-critical or external events
- Using an interrupt is more efficient than polling (waiting) for an event
  - Here “event” is used in a different context; means something that happens outside the task

An interrupt allows hardware to notify the CPU immediately when an event occurs, rather than waiting for software to poll. This is foundational for real-time systems.

Interrupts

- Asynchronous signal from hardware
  - E.g., a GPIO set as an input changes state
- Temporarily suspends normal code execution
- Transfers control to an Interrupt Service Routine (ISR)
- Used for time-critical or external events
- Using an interrupt is more efficient than polling (waiting) for an event
  - Here “event” is used in a different context; means something that happens outside the task

Interrupts are key to providing real-time, low-latency responses

UNIVERSITY OF ALBERTA

Made with help from ChatGPT 22

An interrupt allows hardware to notify the CPU immediately when an event occurs, rather than waiting for software to poll. This is foundational for real-time systems.

Why Interrupts?

Interrupts

- Provide immediate response to external events
- Are lower latency than polling
- Facilitate efficient CPU utilization
- Enable low-power operation
  - Modern microcontrollers can wake a CPU on interrupt
  - RP2040 can wake from deep sleep on GPIO changes or signal from the built-in Real-Time Clock (RTC), which acts as an “alarm clock”

An interrupt allows hardware to notify the CPU immediately when an event occurs, rather than waiting for software to poll. This is foundational for real-time systems.

Interrupts and FreeRTOS

- Interrupts exist outside the FreeRTOS scheduler
- ISR preempts the currently running task
- ISR execution is non-preemptible (by tasks)
- RTOS must be explicitly notified from ISR
- ISRs are not tasks
  - They interrupt task execution
  - Must cooperate with the scheduler via the `FromISR` API
- Keep ISR as short as possible
  - Too long and can interfere with scheduling, e.g., in fixed time-slice schemes

In FreeRTOS, interrupts are not tasks. They interrupt task execution and must cooperate with the scheduler using special “FromISR” APIs.

Interrupt Sources

- Most peripherals generate interrupts
  - Keyboards, mice, other input devices
  - GPIOs (buttons, sensors, encoders)
  - Timers (periodic scheduling, timeouts)
  - Communication peripherals (UART, SPI, I²C)
- We've already seen interrupts used with
  - ADC completion
  - DMA transfer completion

ISRs and FreeRTOS

- Keep ISRs short
- No blocking calls
- No standard FreeRTOS API calls
- Use `FromISR()` functions only
- Can communicate with tasks from ISR
  - Queues (`xQueueSendFromISR`)
  - Semaphores (`xSemaphoreGiveFromISR`)
  - Event groups (`xEventGroupSetBitsFromISR`)
  - Task notifications (`xTaskNotifyFromISR`)

When to use and not use Interrupts

Use interrupts

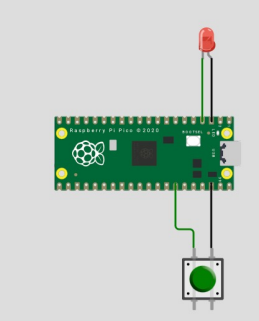
- When there are external asynchronous events
- When there are tight timing requirements
- In low-power designs
- If there are high-frequency or unpredictable events

Don’t use interrupts

- For slow or periodic tasks → use RTOS timers
- If there is complex processing → use tasks
- If there are frequent events with minimal urgency → polling
- When determinism is not required

RP2040 (Pico) Example

- Generate an interrupt from a push button
- Toggle an LED



- On Wokwi
  - ECE 315 Lec 28 FreeRTOS Example

UNIVERSITY OF ALBERTA

```c
/**
 * ECE 315 Computer Interfacing
 *
 * @file main.h
 * @details A simple example to illustrate an Interrupt and ISR
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version   1.0
 * @licence   MIT
 */
#ifndef MAIN_H
#define MAIN_H

// FreeRTOS
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
// C
#include <stdio.h>
// Pico SDK
#include "pico/stdlib.h"
#include "hardware/gpio.h"

#ifdef __cplusplus
extern "C" {
#endif

/**
 * CONSTANTS
 */
#define         RED_LED_PIN           2
#define         BUTTON_PIN           28

#ifdef __cplusplus
}           // extern "C"
#endif


#endif      // MAIN_H
```

Made with help from ChatGPT

RP2040 (Pico) Example

- Set the ISR, `gpio_irq_callback`
  - Check to see if the right GPIO generated the interrupt
  - Good idea in case we change the program later and add other GPIO-triggered interrupts
  - If what we expect, notify the task and yield from the ISR
- The LED task blocks until there is an interrupt, then toggles the LED

```c
/**
 * ECE 315 Computer Interfacing
 *
 * @file main.c
 * @details A simple example to illustrate an Interrupt and ISR
 * @author Steven Knudsen
 * @copyright 2025, University of Alberta
 * @version   1.0
 * @licence   MIT
 */
#include "main.h"

TaskHandle_t led_task_handle = NULL;

// GPIO IRQ callback runs in IRQ context (ISR context)
void gpio_irq_callback(uint gpio, uint32_t events)
{
    if (gpio == BUTTON_PIN && (events & GPIO_IRQ_EDGE_FALL)) {
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;

        // Wake the LED task (increment its notification count)
        vTaskNotifyGiveFromISR(led_task_handle, &xHigherPriorityTaskWoken);

        // Request a context switch if the woken task has higher priority
        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
    }
}

void led_task(void *pvParameters)
{
    (void)pvParameters;

    gpio_init(RED_LED_PIN);
    gpio_set_dir(RED_LED_PIN, GPIO_OUT);

    for (;;) {
        // Block here until ISR notifies us (notification count > 0)
        ulTaskNotifyTake(pdTRUE /* clear count */, portMAX_DELAY);

        // "Deferred work" happens in task context
        gpio_xor_mask(1u << RED_LED_PIN);
    }
}
```

Made with help from ChatGPT

UNIVERSITY OF ALBERTA

RP2040 (Pico) Example

- In main,
  - Initialize the button GPIO
  - Create the LED task
  - Set the IRQ callback, aka ISR
- If you try this online, why is it tricky to get the LED to toggle?
  - How might we fix that problem?

```c
int main(void)
{
    stdio_init_all();
    sleep_ms(200);

    // Configure button input with pull-up and falling-edge interrupt
    gpio_init(BUTTON_PIN);
    gpio_set_dir(BUTTON_PIN, GPIO_IN);
    gpio_pull_up(BUTTON_PIN);

    // Create the task that will be woken by the interrupt
    xTaskCreate(led_task, "LED", 256, NULL, tskIDLE_PRIORITY + 2, &led_task_handle);

    // Register the IRQ callback and enable interrupt
    // Note: callback must be set once; enabling can be done after.
    gpio_set_irq_enabled_with_callback(
        BUTTON_PIN,
        GPIO_IRQ_EDGE_FALL,
        true,
        &gpio_irq_callback
    );

    vTaskStartScheduler();

    // Should never reach here
    while (true) { tight_loop_contents(); }
}
```

Made with help from ChatGPT

Summary

- Real-time operating systems like FreeRTOS emphasize deterministic timing, low latency, and efficient resource usage, with tasks as the primary scheduling units.
- FreeRTOS events and event groups allow tasks to synchronize and block efficiently without busy-waiting, supporting coordination patterns such as task rendezvous.
- Interrupts are asynchronous hardware signals that immediately notify the CPU of external or time-critical events; lower latency and better efficiency than polling.
- In FreeRTOS, Interrupt Service Routines (ISRs) are not tasks and must be short, non-blocking, and use only FromISR() APIs to notify tasks.
- Effective embedded design uses interrupts primarily to signal tasks, deferring substantial processing to task context, as demonstrated by the RP2040 GPIO interrupt example.

Next Lecture

- Analogue/Digital conversion
- ADCs, DACs

Questions

UNIVERSITY OF ALBERTA